

Overhead de Operações Criptográficas em Bancos de Dados Distribuídos

Beatriz Viana Costa¹, Lys Katherine Park Kang¹,
Raphaella Brandão Jacques¹, Stephanie Maria Braga¹

¹Instituto de Matemática e Estatística (IME)
Universidade de São Paulo (USP) – São Paulo, SP – Brasil

Abstract. *As distributed systems increasingly store sensitive data, cryptographic techniques become essential to guarantee confidentiality and integrity. However, these techniques introduce performance costs that are amplified in distributed settings, where inter-node communication is already a natural bottleneck. This work investigates the impact of cryptographic operations on the performance of distributed databases, combining a literature review with practical experiments. We configure a distributed database and measure latency and throughput with and without encryption in transit (TLS), in order to quantify the introduced overhead and discuss the trade-offs between security and performance. Over a real, heterogeneous query workload on MongoDB, enabling client-server TLS introduces a median latency overhead of about 15% (13% on average) and a throughput reduction of about 11%. This cost is dominated by the per-connection handshake: it is nearly constant in absolute terms ($\approx 65\text{--}70$ ms per query), so the relative overhead shrinks as the intrinsic cost of the query grows.*

Resumo. *À medida que sistemas distribuídos passam a armazenar volumes crescentes de dados sensíveis, técnicas criptográficas tornam-se indispensáveis para garantir confidencialidade e integridade. Contudo, essas técnicas introduzem custos de desempenho que se tornam ainda mais relevantes em ambientes distribuídos, nos quais a comunicação entre nós já representa um gargalo natural. Este trabalho investiga o impacto de operações criptográficas sobre o desempenho de bancos de dados distribuídos, combinando uma revisão da literatura com experimentos práticos. Configuramos um banco de dados distribuído e medimos latência e vazão com e sem criptografia em trânsito (TLS), a fim de quantificar o overhead introduzido e discutir os trade-offs entre segurança e desempenho. Sobre uma carga de trabalho real e heterogênea no MongoDB, habilitar o TLS cliente-servidor introduz um overhead mediano de latência de cerca de 15% (13% em média) e uma queda de vazão de cerca de 11%. Esse custo é dominado pelo handshake por conexão: ele é quase constante em termos absolutos ($\approx 65\text{--}70$ ms por consulta), de modo que o overhead relativo diminui conforme cresce o custo intrínseco da consulta.*

1. Introdução

A crescente adoção de bancos de dados distribuídos para o armazenamento de grandes volumes de dados sensíveis torna a proteção criptográfica um requisito central de projeto [Fuller et al. 2017]. Entretanto, garantir confidencialidade e integridade tem um

custo: operações criptográficas consomem CPU e introduzem latência adicional, efeito agravado em sistemas distribuídos, nos quais a comunicação entre nós já constitui um gargalo natural [Gilbert and Lynch 2002].

Este trabalho tem como objetivo quantificar, de forma reproduzível, o overhead introduzido pela criptografia *em trânsito* (TLS) sobre o desempenho de um banco de dados sob uma carga de trabalho real e heterogênea, e discutir o trade-off entre segurança e desempenho. Delimitamos o escopo empírico ao canal de comunicação cliente–servidor de um SGBD orientado a documentos (MongoDB); a criptografia em repouso e em uso é apresentada conceitualmente (Seção 2) e retomada como trabalho futuro (Seção 7), mas não é avaliada empiricamente aqui. Especificamente, buscamos responder:

- **PP1.** Qual é o impacto da criptografia em trânsito (TLS) sobre a latência e a vazão de um SGBD sob uma carga de trabalho real e heterogênea?
- **PP2.** Como esse overhead varia entre diferentes classes de operação (agrupamento, junção, subconsulta, busca, inserção e modificação)?
- **PP3.** Como o overhead *relativo* se relaciona ao custo *intrínseco* de cada consulta?

O restante do artigo está organizado como segue. A Seção 2 apresenta os conceitos fundamentais. A Seção 3 discute o estado da arte. A Seção 4 detalha a metodologia experimental. A Seção 5 apresenta os resultados, discutidos na Seção 6. A Seção 7 conclui o trabalho.

2. Conceitos Fundamentais

Para compreender o impacto de operações criptográficas em bancos de dados distribuídos, é necessário primeiro entender como esses sistemas funcionam, quais mecanismos de criptografia estão disponíveis e como medimos seu desempenho. Esta seção apresenta esses três pilares.

2.1. Bancos de Dados Distribuídos

Um **Banco de Dados Distribuído** (BDD) é uma coleção de bancos de dados logicamente inter-relacionados, fisicamente dispersos por uma rede de computadores e gerenciados por um Sistema de Gerenciamento de Banco de Dados Distribuído (SGBDD) [Elmasri and Navathe 2016]. A motivação para distribuir dados inclui maior disponibilidade, melhor desempenho por proximidade geográfica e escalabilidade horizontal. Contudo, essa distribuição introduz desafios que não existem em sistemas centralizados — em particular, a necessidade de comunicação constante entre nós, que é justamente o canal sobre o qual a criptografia em trânsito atua.

2.1.1. Fragmentação e Replicação

Para distribuir os dados entre os nós, um BDD utiliza duas técnicas complementares: fragmentação e replicação [Elmasri and Navathe 2016].

A **fragmentação** (também chamada de *sharding* ou particionamento) consiste em dividir uma relação em partes menores, cada uma alocada em um nó diferente. Há três abordagens: a *fragmentação horizontal* divide as tuplas segundo algum critério de seleção (por exemplo, por região geográfica); a *fragmentação vertical* distribui os atributos da

relação, como em uma operação de projeção; e a *fragmentação mista* combina ambas as estratégias. A escolha da estratégia influencia diretamente o volume de comunicação entre nós — uma consulta que precisa recombinar fragmentos dispersos gera tráfego de rede, e portanto mais dados a serem cifrados quando TLS está habilitado.

A **replicação**, por sua vez, mantém cópias dos dados em múltiplos nós. Isso melhora a disponibilidade (se um nó falha, outro responde) e o desempenho de leitura (requisições podem ser atendidas localmente). Em contrapartida, escritas precisam ser propagadas para todas as réplicas, gerando tráfego adicional de sincronização. Em um cenário com TLS, cada mensagem de sincronização entre réplicas também passa por cifragem, amplificando o overhead proporcionalmente ao grau de replicação.

2.1.2. Comunicação e Processamento de Consultas

Em um BDD, o processamento de consultas difere fundamentalmente do caso centralizado: o otimizador precisa considerar não apenas o custo de I/O e CPU, mas também o **custo de comunicação** — isto é, o volume de dados transferidos pela rede entre os nós [Elmasri and Navathe 2016]. Estratégias como a semi-junção (*semi-join*) existem justamente para reduzir esse tráfego. Esse custo de comunicação é o ponto central deste trabalho: quando habilitamos TLS, cada byte transmitido entre nós precisa ser cifrado na origem e decifrado no destino, adicionando latência a cada troca de mensagem.

Além disso, para garantir a atomicidade de transações que envolvem múltiplos nós, BDDs utilizam protocolos de coordenação como o *Two-Phase Commit* (2PC), em que um coordenador troca mensagens com todos os participantes para decidir se a transação será confirmada ou abortada [Elmasri and Navathe 2016]. Protocolos de consenso mais modernos, como Raft [Ongaro and Ousterhout 2014], são utilizados em sistemas como Spanner [Corbett et al. 2012]. Todos esses protocolos dependem de múltiplas rodadas de comunicação, o que os torna sensíveis ao overhead de criptografia em trânsito.

2.1.3. Teorema CAP

O teorema CAP [Gilbert and Lynch 2002] formaliza um trade-off fundamental: em um sistema distribuído sujeito a partições de rede, é impossível garantir simultaneamente *Consistência* (todos os nós veem os mesmos dados), *Disponibilidade* (toda requisição recebe resposta) e *Tolerância a Partições*. Como partições são inevitáveis na prática, sistemas precisam optar entre priorizar consistência (modelo CP, como Spanner) ou disponibilidade (modelo AP, como Cassandra). Essa escolha arquitetural determina a quantidade e o padrão de comunicação entre nós, influenciando diretamente o quanto o overhead do TLS se manifesta no desempenho global do sistema.

2.2. Técnicas Criptográficas em Bancos de Dados

A proteção de dados em um banco de dados pode ocorrer em três momentos distintos, de acordo com o estado em que os dados se encontram [Force 2020].

2.2.1. Criptografia em Repouso (*at rest*)

Quando os dados estão armazenados em disco, a criptografia em repouso protege contra acessos não autorizados ao meio físico (roubo de disco, acesso indevido a arquivos do sistema operacional). A abordagem mais comum é a *Transparent Data Encryption* (TDE), que cifra as páginas de dados e os arquivos de log de forma transparente para a aplicação — ou seja, o banco realiza a cifragem e decifragem automaticamente nas operações de I/O, sem alterar as consultas [Force 2020].

O algoritmo predominante é o AES (*Advanced Encryption Standard*), um cifrador de bloco simétrico padronizado pelo NIST [National Institute of Standards and Technology (NIST) 2001], com chaves de 128, 192 ou 256 bits. A granularidade pode variar: criptografia de todo o disco, por *tablespace*, por tabela ou mesmo por coluna individual. O overhead da criptografia em repouso incide sobre cada operação de leitura e escrita em disco, pois os dados precisam ser decifrados ao serem carregados para a memória e cifrados novamente ao serem persistidos.

2.2.2. Criptografia em Trânsito (*in transit*)

A criptografia em trânsito protege os dados enquanto trafegam pela rede — seja entre um cliente e o servidor, seja entre os nós internos do cluster distribuído. O protocolo padrão é o TLS (*Transport Layer Security*) [Rescorla 2018], atualmente na versão 1.3.

O custo de desempenho do TLS pode ser decomposto em dois componentes:

1. **Handshake:** no estabelecimento da conexão, cliente e servidor negociam parâmetros criptográficos utilizando criptografia assimétrica (RSA ou curvas elípticas). É uma operação computacionalmente cara, mas realizada apenas uma vez por conexão (ou por reconexão).
2. **Cifragem simétrica contínua:** após o handshake, todos os dados são cifrados com algoritmos simétricos como AES-GCM ou ChaCha20-Poly1305. O custo por pacote é baixo individualmente, mas em cenários de alta vazão (muitas operações por segundo ou grandes volumes de dados) o impacto acumula.

Em um banco de dados distribuído, o TLS pode ser habilitado tanto na comunicação cliente–servidor quanto na comunicação interna entre os nós do cluster (*intra-cluster*). Neste segundo caso, o impacto tende a ser mais expressivo, pois protocolos de replicação e consenso geram tráfego constante entre os nós.

2.2.3. Criptografia em Uso (*in use*)

Uma linha de pesquisa mais recente busca permitir computações diretamente sobre dados cifrados, eliminando a necessidade de decifrá-los. As abordagens incluem: criptografia determinística e que preserva ordem (OPE) [Boldyreva et al. 2009], que viabilizam consultas de igualdade e intervalos sobre dados cifrados, como explorado pelo sistema CryptDB [Popa et al. 2011]; criptografia totalmente homomórfica (FHE) [Gentry 2009],

que permite computações arbitrárias mas com overhead ainda proibitivo para cargas transacionais; e soluções baseadas em enclaves seguros (*trusted execution environments*), como o Always Encrypted [Antonopoulos et al. 2020]. Embora não sejam o foco dos nossos experimentos, essas técnicas representam o estado da arte em proteção de dados e servem como horizonte para trabalhos futuros.

2.2.4. Integridade

Complementando a confidencialidade, mecanismos de integridade garantem que os dados não foram corrompidos ou adulterados. Técnicas como *hashing* criptográfico (SHA-256), *HMAC* (Hash-based Message Authentication Code) e árvores de Merkle permitem detectar alterações tanto em dados armazenados quanto em dados em trânsito [Force 2020]. No contexto do TLS, a integridade é garantida implicitamente pelos modos de operação AEAD (*Authenticated Encryption with Associated Data*), como o AES-GCM, que combina cifragem e autenticação em uma única operação.

2.3. Métricas de Desempenho

Para avaliar o impacto das operações criptográficas de forma rigorosa, é necessário definir as métricas que serão observadas [Cooper et al. 2010]:

- **Latência:** o tempo entre o envio de uma requisição e o recebimento da resposta correspondente. Como a distribuição de latências tende a ser assimétrica (com uma “cauda longa”), reportamos não apenas a média, mas também a mediana (p50) e os percentis p95 e p99, que revelam o comportamento nos piores casos.
- **Vazão (*throughput*):** o número de operações completadas por unidade de tempo (operações/segundo). Enquanto a latência captura a experiência individual de cada requisição, a vazão reflete a capacidade agregada do sistema sob carga.
- **Utilização de CPU:** a fração do processamento dedicada a operações criptográficas. Permite distinguir se o overhead observado decorre de cifragem ou de outros fatores (I/O, contenção de rede).
- **Overhead de rede:** o aumento no volume de bytes transmitidos devido aos cabeçalhos e metadados introduzidos pelo TLS (registros TLS, MACs, padding).

Para quantificar o impacto relativo da criptografia, definimos o **overhead** em relação ao cenário *baseline* (sem criptografia):

$$\text{overhead}(\%) = \frac{M_{\text{cripto}} - M_{\text{baseline}}}{M_{\text{baseline}}} \times 100 \quad (1)$$

onde M representa o valor de uma métrica (latência ou vazão) no respectivo cenário. Um overhead positivo em latência indica degradação; um overhead negativo em vazão indica queda na capacidade do sistema. Neste trabalho, as métricas efetivamente coletadas são a **latência** (com ênfase na mediana, p50) e a **vazão**; a utilização de CPU e o overhead de rede são definidos acima por completude conceitual, mas não foram instrumentados em nosso arcabouço de medição (Seção 4).

3. Trabalhos Relacionados

Esta seção situa nosso estudo em relação a (i) trabalhos teóricos e de sistemas sobre proteção criptográfica de bancos de dados e (ii) ferramentas de software livre relevantes para o experimento.

3.1. Estudos Teóricos e de Sistemas

Boa parte da literatura sobre criptografia em bancos de dados concentra-se na proteção dos dados *em repouso* e *em uso*, buscando permitir consultas diretamente sobre dados cifrados. O CryptDB [Popa et al. 2011] foi um marco nessa direção: combinando criptografia determinística e que preserva ordem (OPE) [Boldyreva et al. 2009], executa consultas SQL de igualdade e intervalo sobre dados cifrados, reportando um overhead de vazão da ordem de 20–30% em cargas transacionais. O *Always Encrypted* [Antonopoulos et al. 2020] adota abordagem complementar, apoiando-se em enclaves seguros (*trusted execution environments*) para processar dados decifrados apenas dentro de uma região protegida do processador. No extremo da expressividade está a criptografia totalmente homomórfica (FHE) [Gentry 2009], que admite computações arbitrárias sobre cifras, mas cujo custo ainda é proibitivo para cargas de produção. A systematization of knowledge de Fuller et al. [Fuller et al. 2017] organiza esse espaço de projeto, evidenciando o trade-off recorrente entre expressividade das consultas, vazamento de informação e desempenho.

Em contraste, a criptografia *em trânsito* (TLS) [Rescorla 2018] costuma ser tratada como um custo secundário e “barato”, raramente quantificado de forma sistemática na literatura de *benchmarking* de SGBDs. Nosso trabalho examina justamente essa suposição, medindo empiricamente o custo do TLS no canal cliente–servidor de um SGBD orientado a documentos.

3.2. Ferramentas de Software Livre

Conforme sugerido no enunciado, priorizamos ferramentas de software livre, que podem ser instaladas e testadas sem restrições. No que diz respeito a SGBDs, o **MongoDB Community** oferece suporte nativo a TLS tanto no canal cliente–servidor quanto na comunicação intra-cluster [MongoDB, Inc. 20XX], o que o torna adequado ao nosso experimento. Entre os SGBDs relacionais distribuídos (*NewSQL*), CockroachDB e YugabyteDB também oferecem TLS intra-cluster e criptografia em repouso, enquanto o Apache Cassandra segue o modelo AP do teorema CAP [Gilbert and Lynch 2002]. A cifragem em si, em todos esses sistemas, é tipicamente delegada a bibliotecas como o OpenSSL e acelerada por instruções de hardware (AES-NI).

Para a geração de carga, ferramentas consolidadas incluem o YCSB [Cooper et al. 2010], voltado a sistemas NoSQL, além de sysbench e BenchBase (TPC-C). Optamos, contudo, por uma carga composta de *consultas reais* herdadas de um projeto anterior (Seção 4.2), em vez de um gerador sintético: essa escolha sacrifica a padronização típica dos geradores em favor de maior validade ecológica, capturando padrões de acesso heterogêneos (analíticos e transacionais).

3.3. Posicionamento

Os *benchmarks* clássicos tendem a enfatizar a criptografia em repouso ou consultável sobre cargas sintéticas e uniformes, nas quais todas as operações têm custo semelhante. Nosso estudo difere em dois pontos. Primeiro, isola o canal *em trânsito* (TLS) de um repositório de documentos sob uma mistura heterogênea de consultas reais — analíticas (agrupamento, junção, subconsulta) e transacionais (busca, inserção, modificação).

Segundo, caracteriza como o overhead *relativo* depende do custo *intrínseco* de cada consulta — dimensão que cargas sintéticas uniformes tendem a ocultar, mas que se mostra central para interpretar corretamente o impacto da criptografia.

4. Metodologia

O método empregado neste estudo é o **experimento** controlado. Descrevemos a seguir, com o detalhe necessário à reprodução, o ambiente, a carga de trabalho, os cenários e o protocolo de medição. Todo o código e os dados citados encontram-se no repositório do projeto.

4.1. Ambiente Experimental

Os experimentos foram executados em um único host — um computador _____ (processador _____, _____ GB de RAM) com sistema operacional _____. A orquestração é feita via **Docker Compose**, que instancia dois SGBDs **MongoDB** a partir da imagem oficial `mongo` (versão _____):

- `mongo_baseline` — na porta 27017, sem qualquer criptografia (configuração padrão);
- `mongo_tls` — na porta 27018, iniciado com `--tlsMode requireTLS` e um par certificado/chave *self-signed*, exigindo TLS mas não a autenticação mútua por certificado de cliente (`--tlsAllowConnectionsWithoutCertificates`).

O cliente utilizado é o `mongosh` (*MongoDB Shell*), invocado pelo script de automação. Como ambos os contêineres residem no mesmo host, a comunicação cliente-servidor ocorre via *loopback*: dessa forma, o que medimos é predominantemente o **custo de CPU** da criptografia (handshake e cifra), e não a latência de uma rede de longa distância. Um terceiro contêiner (PostgreSQL 15) foi usado apenas para extrair e converter o conjunto de dados (Seção 4.2), não participando das medições.

É importante delimitar o escopo: trata-se de *instâncias de nó único*, sem fragmentação (*sharding*) nem replicação (*replica set*). Medimos, portanto, o canal cliente-servidor, e não a amplificação criptográfica da comunicação *intra-cluster* (replicação e consenso). Essa limitação é retomada na Seção 6.

4.2. Carga de Trabalho

Conjunto de dados. Utilizamos o conjunto de dados público do *Stack Exchange Data Dump* [Stack Exchange, Inc. 20XX], herdado de um projeto anterior da disciplina. Originalmente relacional, o dump foi extraído de um PostgreSQL e convertido em coleções MongoDB, carregadas de forma *idêntica* nos dois contêineres — o que é essencial para uma comparação justa. A Tabela 1 resume o volume de dados. Os mesmos índices secundários foram criados em ambas as instâncias (por exemplo, sobre `Id`, `OwnerUserId`, `PostId`, `UserId`, `Name`, `ViewCount` e `Score`).

Consultas. Em vez de um gerador sintético, a carga é composta de **30 consultas reais** (também herdadas do projeto anterior), organizadas em seis classes que cobrem tanto operações analíticas quanto transacionais. A Tabela 2 descreve as classes e sua quantidade. As mesmas 30 consultas, na mesma ordem, são executadas em ambos os cenários.

Tabela 1. Volume do conjunto de dados (coleções MongoDB), idêntico nos dois cenários.

Coleção	Documentos
Posts	500 000
Comments	500 000
Badges	500 000
Votes	500 000
Users	299 398
PostLinks	93 091
PostTypes	8

Tabela 2. Classes de consulta que compõem a carga de trabalho (30 consultas ao todo).

Classe	Descrição / exemplo	Qtd.
Agrupamento	Agregações analíticas (\$group, \$avg, \$sum)	6
Junção	Junções entre coleções (\$lookup, até 4 níveis)	7
Subconsulta	Filtros correlacionados (\$expr, pipelines internos)	5
Busca	Busca por chave e por padrão textual (find, \$regex)	2
Inserção	Inserções individuais e em lote (insertOne, insertMany)	5
Modificação	Atualizações e remoções (updateMany, deleteMany)	5
Total		30

O cliente é *único* e as consultas são executadas de forma *sequencial* (uma repetição por vez), de modo que a vazão medida reflete a capacidade sob um cliente serial, e não sob concorrência (ver Seção 4.4).

4.3. Cenários Avaliados

Avaliamos dois cenários, que diferem *exclusivamente* quanto à criptografia em trânsito no canal cliente–servidor (Tabela 3). Manter tudo o mais constante (host, dataset, índices, consultas e ordem de execução) é o que permite atribuir a diferença de desempenho ao TLS. A criptografia em repouso (TDE) e em uso, embora conceitualmente relevantes (Seção 2), **não são avaliadas empiricamente** neste estudo e ficam como trabalho futuro (Seção 7).

Tabela 3. Cenários avaliados. Diferem apenas quanto à criptografia em trânsito (TLS).

Cenário	Em trânsito (TLS)	Configuração
C0 (baseline)	não	mongod padrão, porta 27017
C1 (TLS)	sim	mongod com requireTLS, porta 27018

4.4. Protocolo de Medição

A medição é orquestrada por um script de automação. Para *cada uma* das 30 consultas e *cada* cenário, o procedimento é:

1. **Aquecimento (*warm-up*):** uma execução descartada, que aquece o *cache* do sistema operacional e o motor de armazenamento WiredTiger;
2. **Medição:** 20 execuções cronometradas.

Cada execução consiste em invocar o `mongosh` como um *novo processo*, que abre uma *nova conexão* ao `mongod`, executa a consulta a partir de um arquivo `.js` e encerra. Cronometra-se o tempo de parede (*wall-clock*) da invocação completa. A partir das 20 amostras, calculam-se, por consulta e cenário: a latência mediana (p50), os percentis de cauda (p95, p99) e a vazão, definida como $n / \sum_i t_i$ — isto é, o *recíproco da latência média* sob um único cliente sequencial. O overhead relativo é obtido pela Equação (1), sempre em relação ao baseline (C0).

Três características do arcabouço merecem registro explícito, por afetarem a interpretação dos resultados:

- **Conexão nova por execução.** Como cada repetição abre uma conexão nova, no cenário C1 o *handshake* TLS é pago a *cada* execução. Isso corresponde ao *pior caso* para o custo do handshake — ou seja, a ausência de reúso de conexões (*connection pooling*).
- **Partida do `mongosh`.** O tempo absoluto inclui a inicialização do processo `mongosh` (ambiente Node.js), o que infla os valores absolutos de latência (é por isso que mesmo a consulta mais barata leva $\approx 0,43$ s). Contudo, essa parcela é *comum* aos dois cenários e, portanto, *cancela-se na diferença* baseline–TLS. Assim, embora os valores absolutos não representem a latência pura da consulta, a *diferença* entre os cenários isola legitimamente o custo do TLS (handshake e cifra).
- **Estimadores de cauda com $n = 20$.** Os índices usados para p95 e p99 ($\lfloor 20 \cdot 0,95 \rfloor = \lfloor 20 \cdot 0,99 \rfloor = 19$) coincidem, de modo que p95 e p99 correspondem *ambos ao máximo* observado — razão pela qual são idênticos em nossos dados. Por isso, adotamos a **mediana (p50)** como métrica principal de latência e tratamos p95/p99 apenas como um indicador de cauda (o valor máximo), com essa ressalva.

A reprodutibilidade é favorecida pelo caráter determinístico do experimento: o mesmo dataset é carregado em ambas as instâncias, as 30 consultas são fixas e executadas na mesma ordem nos dois cenários, e o código de automação está disponível no repositório.

5. Experimentos e Resultados

Habilitar o TLS degradou o desempenho em *todas* as 30 consultas, sem exceção. Considerando o conjunto completo, o overhead de latência (p50) foi de **13,0% em média** (mediana de 15,4%), variando de 2,6% a 17,1% conforme a consulta; a vazão caiu, correspondentemente, **11,1% em média** (mediana de 13,1%). A Tabela 4 agrega esses resultados por classe de consulta.

Latência e vazão por classe. A Figura 1 contrasta a latência p50 média de cada classe nos dois cenários (em escala logarítmica, dada a grande dispersão entre consultas leves e pesadas), e a Figura 2 faz o mesmo para a vazão. Em ambas, a barra do cenário TLS

Tabela 4. Overhead do TLS por classe de consulta (média das consultas de cada classe). Overhead de latência calculado sobre a p50; queda de vazão em relação ao baseline.

Classe	Consultas	Overhead p50 (%)	Queda de vazão (%)
Agrupamento	6	11,3	11,2
Junção	7	13,4	11,3
Subconsulta	5	12,2	10,2
Busca	2	15,9	13,9
Inserção	5	15,7	12,9
Modificação	5	11,7	8,5
Geral	30	13,0	11,1

é consistentemente pior, mas a distância entre as barras é pequena e razoavelmente uniforme — um primeiro indício de que o custo do TLS é aproximadamente *aditivo*, e não proporcional ao trabalho da consulta.

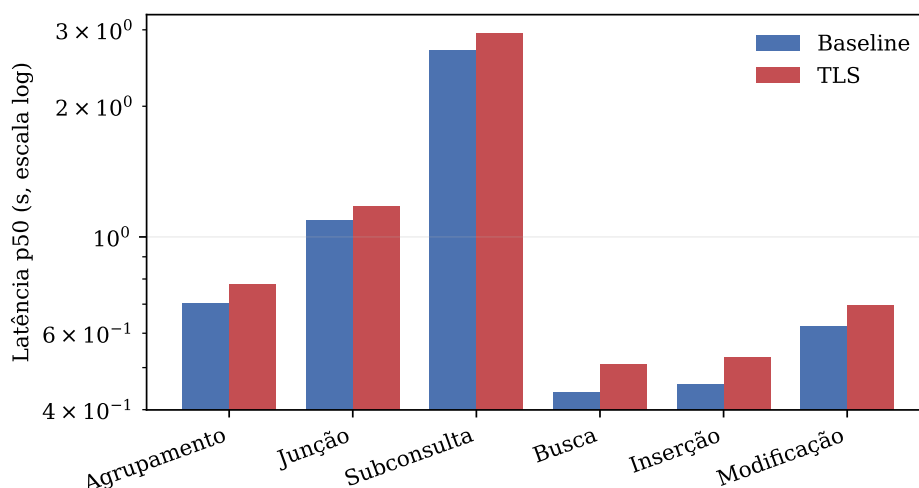


Figura 1. Latência p50 média por classe de consulta, baseline *versus* TLS (escala logarítmica no eixo vertical).

O custo do TLS é quase constante. A observação central do estudo aparece ao examinar o overhead em termos *absolutos*: a diferença de latência p50 entre TLS e baseline é notavelmente estável, com **mediana de 71,7 ms** por consulta (mínimo de 62,7 ms). Esse valor quase fixo é a assinatura do *handshake* TLS pago a cada conexão (Seção 4.4): como cada execução abre uma conexão nova, o custo de estabelecimento do canal seguro incide uma vez por consulta, praticamente independente do trabalho que a consulta realiza.

A consequência direta é que o overhead *relativo* não é uniforme — ele *diminui* conforme cresce o custo intrínseco da consulta, pois o custo fixo do *handshake* é amortizado por um tempo de execução maior. A Figura 3 torna esse efeito explícito: consultas baratas (busca, inserção, junções e subconsultas simples), com p50 baseline em torno de

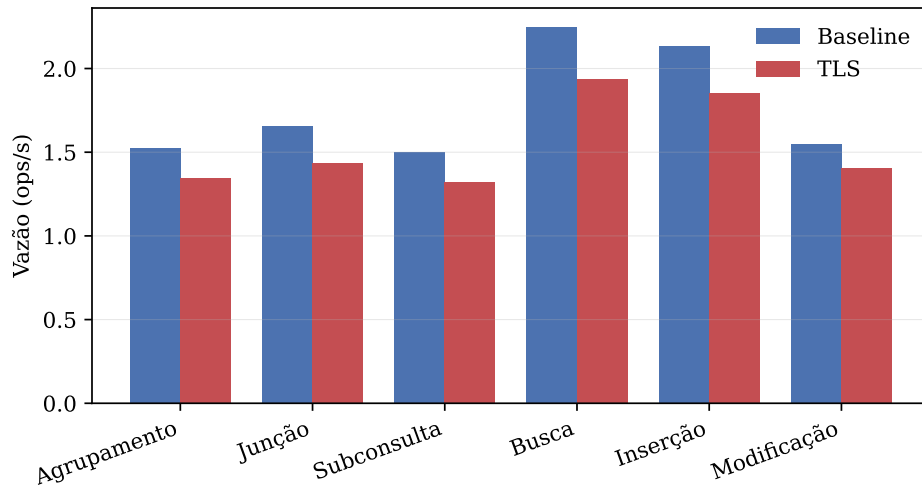


Figura 2. Vazão média (operações por segundo) por classe de consulta, baseline versus TLS.

0,4–0,5 s, sofrem overhead de 15–17%, ao passo que as duas consultas mais custosas — uma junção de múltiplos `lookup` (p50 baseline de 4,2 s) e uma subconsulta correlacionada (11,1 s) — caem para 2,6% e 9,0%, respectivamente. Em outras palavras, o *mesmo* custo absoluto de segurança representa uma fração cada vez menor do tempo total à medida que a consulta se torna mais pesada.

Resultados detalhados. A Tabela 5 apresenta, para as 30 consultas, a latência p50 nos dois cenários, o overhead relativo e a queda de vazão.

6. Discussão

6.1. O trade-off segurança versus desempenho

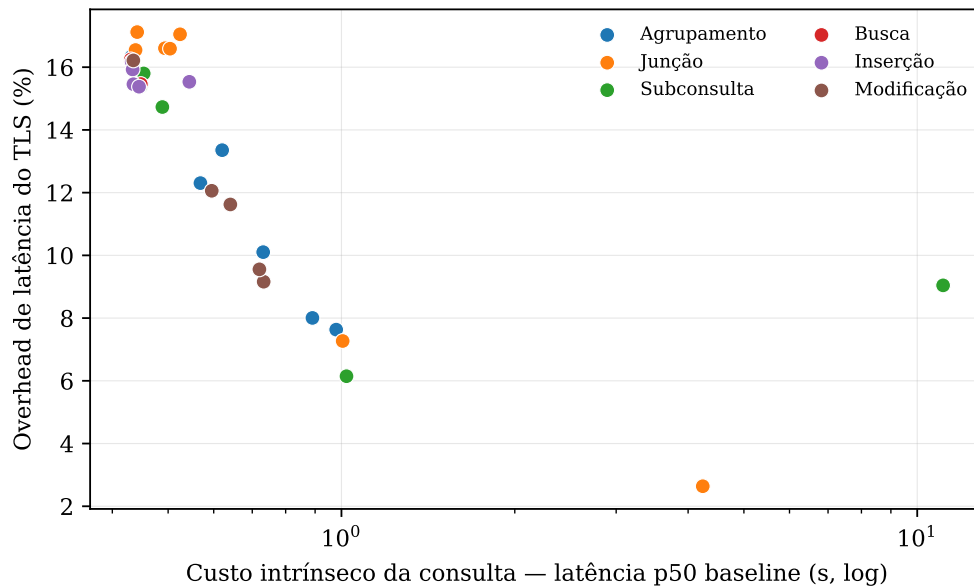
Os resultados sugerem que, no canal cliente–servidor, o custo do TLS é *moderado e previsível*. Um overhead mediano em torno de 13–15% em latência e de 11–13% em vazão é, para a maioria das aplicações, um preço aceitável pela confidencialidade e integridade que o TLS proporciona — especialmente diante de exigências regulatórias como a LGPD e o GDPR. Mais importante do que o número absoluto é o seu *padrão*: por ser aproximadamente aditivo, o custo do TLS é fácil de prever e de planejar em termos de capacidade.

6.2. Qual componente domina o overhead?

A evidência aponta de forma consistente para o **handshake por conexão** como componente dominante, e não para a cifragem simétrica contínua dos dados. Dois fatos sustentam essa conclusão: (i) o custo absoluto adicional é quase constante (mediana de 71,7 ms), independentemente do volume de dados que a consulta processa ou retorna; e (ii) o overhead relativo decresce monotonicamente com o custo da consulta (Figura 3). Se a cifra simétrica sobre os dados transferidos fosse o fator dominante, esperaríamos o oposto — consultas que retornam mais dados pagariam *mais*, não menos. A cifra em si, acelerada por instruções de hardware (AES-NI) e barata por *byte*, é desprezível diante do custo assimétrico do estabelecimento da conexão para os tamanhos de resultado desta carga.

Tabela 5. Resultados por consulta: latência p50 (s) no baseline e com TLS, overhead de latência (%) e queda de vazão (%).

Classe	#	p50 base (s)	p50 TLS (s)	Overhead (%)	Queda vazão (%)
Agrupamento	1	0,979	1,054	7,6	10,9
Agrupamento	2	0,731	0,805	10,1	9,2
Agrupamento	3	0,621	0,703	13,4	15,1
Agrupamento	4	0,432	0,502	16,3	14,1
Agrupamento	5	0,890	0,962	8,0	7,5
Agrupamento	6	0,569	0,639	12,3	10,6
Junção	1	4,241	4,353	2,6	2,4
Junção	2	0,439	0,512	16,5	15,0
Junção	3	0,524	0,614	17,0	13,1
Junção	4	0,494	0,576	16,6	14,1
Junção	5	0,504	0,587	16,6	14,3
Junção	6	0,442	0,517	17,1	13,5
Junção	7	1,005	1,078	7,3	6,6
Subconsulta	1	0,453	0,525	15,8	13,4
Subconsulta	2	1,020	1,083	6,1	4,1
Subconsulta	3	11,090	12,093	9,0	6,7
Subconsulta	4	0,448	0,518	15,4	13,5
Subconsulta	5	0,489	0,561	14,7	13,3
Busca	1	0,431	0,501	16,2	13,9
Busca	2	0,448	0,518	15,5	13,9
Inserção	1	0,432	0,502	16,2	14,5
Inserção	2	0,434	0,503	15,9	13,1
Inserção	3	0,435	0,502	15,5	19,7
Inserção	4	0,445	0,514	15,4	6,9
Inserção	5	0,544	0,629	15,5	10,3
Modificação	1	0,595	0,667	12,1	5,4
Modificação	2	0,641	0,716	11,6	9,1
Modificação	3	0,435	0,506	16,2	15,1
Modificação	4	0,733	0,800	9,2	8,5
Modificação	5	0,720	0,789	9,6	4,5



Validade externa. O experimento roda em *nó único* e por *loopback*. Isso significa que (i) não medimos a amplificação criptográfica da comunicação *intra-cluster* — em um *replica set* ou cluster fragmentado, cada escrita replicada e cada rodada de consenso também tráfegaria cifrada, tendendo a ampliar o overhead em relação ao que observamos; e (ii) a ausência de latência de rede real faz com que a fração de tempo atribuível à criptografia (custo de CPU) seja relativamente maior do que seria sobre uma WAN. Além disso, avaliamos apenas a criptografia *em trânsito*: os custos de criptografia *em repouso* (TDE) e *em uso* não foram medidos. O certificado é *self-signed* e a autenticação mútua está desabilitada, de modo que não contabilizamos o custo de validação de cadeias de certificados nem de mTLS, presentes em ambientes de produção. Finalmente, os resultados referem-se a um único conjunto de dados (Stack Exchange) e a um único host; cargas, esquemas e hardware distintos podem deslocar os números.

7. Conclusão e Trabalhos Futuros

Este trabalho quantificou, de forma reproduzível, o overhead da criptografia em trânsito (TLS) sobre o desempenho do MongoDB, sob uma carga de 30 consultas reais e heterogêneas. Retomando as perguntas de pesquisa:

- **PP1.** Habilitar o TLS no canal cliente–servidor degradou o desempenho em todas as consultas, com overhead mediano de latência de cerca de 15% (13% em média) e queda de vazão de cerca de 13% (11% em média) — um custo moderado e previsível.
- **PP2.** O overhead variou entre as classes (de ~11% em agrupamento/modificação a ~16% em busca/inserção), mas essa variação *não* decorre da semântica da classe: decorre do custo típico das consultas de cada classe.
- **PP3.** O overhead *relativo* é inversamente proporcional ao custo intrínseco da consulta. Isso porque o custo do TLS é quase constante em termos absolutos (~65–70 ms), dominado pelo *handshake* pago a cada conexão; quanto mais cara a consulta, mais esse custo fixo se dilui.

A mensagem prática é dupla: o TLS em trânsito é barato o bastante para ser a opção padrão na maioria dos cenários; e o reúso de conexões (*connection pooling*) é a alavanca mais eficaz para reduzir seu custo, já que ataca diretamente o componente dominante.

Trabalhos futuros. As limitações apontadas na Seção 6 delineiam extensões naturais. Primeiro, avaliar a criptografia *em repouso* (por exemplo, a cifragem no nível do WiredTiger ou via LUKS no volume de dados), completando os cenários originalmente previstos. Segundo, migrar de instâncias de nó único para um *replica set* ou cluster fragmentado, a fim de medir a amplificação do TLS na comunicação intra-cluster (replicação e consenso). Terceiro, um estudo específico do efeito do *connection pooling*, contrastando o pior caso aqui medido com o custo amortizado. Por fim, avançar para a criptografia *em uso*, avaliando recursos como a *Queryable Encryption* do próprio MongoDB, e replicar o experimento em outros SGBDs distribuídos e sobre uma rede real.

Referências

Antonopoulos, P., Arasu, A., Singh, K. D., Eguro, K., Gupta, N., Jain, R., Kaushik, R., Kodavalla, H., Kossmann, D., Ramamurthy, R., Szymaszek, J., Trimmer, J., Vaswani,

- K., Venkatesan, R., and Zwillig, M. (2020). Azure SQL database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525. ACM.
- Boldyreva, A., Chenette, N., Lee, Y., and O’Neill, A. (2009). Order-preserving symmetric encryption. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 224–241. Springer.
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., and Sears, R. (2010). Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM.
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., et al. (2012). Spanner: Google’s globally-distributed database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 261–264. USENIX Association.
- Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, 7 edition.
- Force, J. T. (2020). Security and privacy controls for information systems and organizations. Special Publication SP 800-53 Rev. 5, National Institute of Standards and Technology (NIST). <https://doi.org/10.6028/NIST.SP.800-53r5>.
- Fuller, B., Varia, M., Yerukhimovich, A., Shen, E., Hamlin, A., Gadepally, V., Shay, R., Mitchell, J. D., and Cunningham, R. K. (2017). SoK: Cryptographically protected database search. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 172–191. IEEE.
- Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178. ACM.
- Gilbert, S. and Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59.
- MongoDB, Inc. (20XX). Configure mongod and mongos for TLS/SSL. MongoDB Manual. <https://www.mongodb.com/docs/manual/tutorial/configure-ssl/>.
- National Institute of Standards and Technology (NIST) (2001). Advanced encryption standard (AES). Federal Information Processing Standards Publication FIPS PUB 197, NIST.
- Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pages 305–319. USENIX Association.
- Popa, R. A., Redfield, C. M. S., Zeldovich, N., and Balakrishnan, H. (2011). CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, pages 85–100. ACM.
- Rescorla, E. (2018). The transport layer security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc8446>.

Stack Exchange, Inc. (20XX). Stack exchange data dump. Internet Archive. <https://archive.org/details/stackexchange>.